Enhancing Robustness in Pretrained Language Models via Model-Level Debiasing

Aidan Sanchez-Cooney
University of Texas at Austin
aidancooney@utexas.edu

Abstract

Research on modeling natural language inference tasks, such as entailment classification, has made massive progress in recent years. Access to large pretrained models vastly improves the capabilities of natural language processing solutions, as these models can be fine-tuned on a particular task and leverage the large embeddings they are already trained on. These large models can exhibit impressive performance on a variety of natural language tasks; however, if not properly evaluated and debiased, they can learn biases in the training data and create shortcuts to classification labels, called artifacts. This paper walks through one such example, using the ELECTRA-Discriminator model to expose the pretrained model's reliance on artifacts to predict entailment. Using a set of CheckList examples, I evaluated the base ELECTRA model against a debiased solution that used the same ELECTRA model as its base. Due to the increased complexity of the feature space for the debiased model, I observed performance gains compared to the baseline model on many tests but encountered drawbacks on a handful of more difficult examples. Further tuning for the debiasing architecture could improve the resulting model's overall robustness on these challenging examples.

1 Introduction

1.1 Background

Pretrained models are effective tools for natural language inference (NLI) tasks. Using the transformer architecture (Vaswani et al., 2023), these models can generate robust embeddings that users can fine-tune for more specific tasks, such as entailment prediction. These models can achieve high validation accuracies when trained on a sufficiently large and quality corpus of examples. While these models are extremely powerful, during fine-tuning, they can pick up biases in the data that lead to shortcuts to a particular classification label. These learned keyword or phrase shortcuts, called artifacts, inflate the model's performance on the training and validation splits but cause it to struggle to generalize well to out-of-test data. The model's reliance on these artifacts generates uncertainty about what is actually being learned during training.

This paper evaluates one of these pretrained models, fine-tuned on a large corpus of natural language inference data, by using CheckList sets (Ribeiro et al., 2020) to identify artifacts the model is exploiting and to assess the impact of debiasing techniques on improving model performance on out-of-test data. Using the ELECTRA-Small-Discriminator model, I fine-tuned it on the Stanford Natural Language Inference (SNLI) corpus for the task of entailment classification. Entailment classification involves evaluating a premise and a hypothesis to determine if the hypothesis is supported by the premise. The SNLI data is labeled with entailment, neutral, or contradiction. The entailment label is assigned when the premise supports the hypothesis, neutral is assigned when the hypothesis is neither supported nor refuted by the premise, and contradiction is assigned when the premise refutes the hypothesis.

Once trained, I evaluated the model on a set of handcrafted examples designed to stress-test its prediction capabilities. Using templates to generate samples for the CheckList examples, I attempted to target specific areas where the model could exploit artifacts learned in training. To improve model performance on these CheckList examples, I trained a bag-of-words (BOW) sub-model alongside the main model to reduce the bias learned by the main model. I then evaluated the new debiased model to assess the effectiveness of these changes, comparing its overall robustness with that of the baseline ELECTRA model.

Artifact	Examples
Negation	No, Not, Cannot ...
Lexical Overlap	*P:* The man is running. *H:* The man is reading.
Universal Qualifiers	Everybody, Anybody, ...

Table 1: Three artifacts that the ELECTRA model will be evaluated on. For the Lexical Overlap example P: Premise and H: Hypothesis. The Lexical Overlap tests are designed to elicit the entailment label by being similar but containing differences in action.

1.2 Stanford NLI Dataset

The dataset used to fine-tune the ELECTRA model is the Stanford Natural Language Inference corpus, a large annotated corpus of premise-hypothesis pairings (Bowman et al., 2015). The SNLI dataset was created using a pre-existing corpus for the premises and prompting workers to generate hypothesis sentences for entailment, neutral, and contradiction labels simultaneously, resulting in a class-balanced dataset. These workers generated over 550,000 samples designed for training and fine-tuning large transformer models on NLI tasks.

1.3 Dataset Artifacts

Dataset artifacts are a common pitfall when evaluating large transformer-based neural models on natural language inference tasks, often inflating performance metrics. These artifacts are elements within the training data that the model learns to associate with a particular label, creating shortcuts to that label, which the model can then use in evaluation. In this paper, I target several common artifacts, including negation terms, lexical overlap, and universal qualifiers. Table 1 contains examples of some of these artifacts.

2 Checklist Sets

2.1 Generating the Checklist Tests

Evaluating an NLI model beyond the train and test splits is often difficult, but a critical step to ensure the model generalizes well. The CheckList method (Ribeiro et al., 2020) provides an outline of testing various capabilities the model should possess. The three main tests it outlines are the Minimum Functionality Tests (MFT), Invariance Tests (INV), and the Directional Expectation Tests (DIR). To generate examples for these tests, I developed templates to capture several test areas and used a script to fill them, generating as diverse a test space as possible with limited resources. The tests I generated for each category include basic tests, as well as artifact tests that use words and phrases designed to expose the model's use of artifacts (Table 1). I split these tests by target label to perform a within-group analysis of the model to identify which label-artifact pair it struggles with most.

The artifact test sets are where the CheckList set can expose the shortcuts the model learned in training. For the entailment label, I generated examples using negation terms to determine if the model associated those negative words and phrases with the contradiction label. For the neutral label, I used a combination of all the artifacts I tested for. Finally, for the contradiction label, I implemented high lexical overlap and universal qualifiers to determine if the model associated them with the entailment label.

The fundamental idea behind the MFT tests is to assess the model's basic functionality. To generate the MFT samples, I used a basic template containing a single subject and a single verb in the form *"A {subject} is {verb}ing"*, filling in the subject and verb from a list of subject-verb pairs to ensure the verb is reasonable given the particular selected subject. To generate the artifact sets I altered these templates slightly to generate double negations for the entailment examples and created contradiction examples where the premise specified a single subject doing an action exclusively while the hypothesis employed a universal qualifier in the form ***Premise:*** *Only the {subject} is {verb}ing.* ***Hypothesis:*** *Everybody is {verb}ing*. For neutral examples, I created pairs with different verbs, ensuring that each activity is distinct but not mutually exclusive. By using a handful of templates similar in form to this and adding more artifact samples with lexical overlap, as shown in Table 1, I generated a variety of unique samples designed to test the minimum functionality of an NLI model.

The INV tests were designed to test the model's ability to understand a sentence's semantics by perturbing the hypothesis in a way that would not change the resulting label. The INV tests were generated using the exact templates used for the MFT tests, with slight modifications. Instead of containing a single premise-hypothesis pairing, the INV tests contained two hypotheses: the base hypothesis, which mirrored that of the MFT test, and a perturbed hypothesis that contained the alteration designed not to change the meaning of the hypothesis. These alterations include synonym swapping

Type	Label	Basic Acc	Artifact Acc	Noise Acc	Antonym Acc
MFT	Entail	85.7	18.8	—	—
	Neutral	45.0	29.5	—	—
	Contradict	98.4	87.3	—	—
INV	Entail	95.4	37.7	46.9	—
	Neutral	9.9	6.7	2.5	—
	Contradict	23.9	70.9	33.6	—
DIR	Entail-Neutral	10.0	0.5	—	—
	Entail-Contradict	67.9	45.5	—	15.7
	Neutral-Entail	1.1	0.1	—	—
	Neutral-Contradict	0.7	0.2	—	—
	Contradict-Entail	59.7	3.8	—	4.5
	Contradict-Neutral	5.7	0.1	—	—

Table 2: Analysis of the ELECTRA discriminator on two main sets of tests: basic checklist sets and artifact checklist sets. The addition of Noise and Antonym tests are added for a select few categories where applicable.

to create examples in the form ***Premise:*** *{subject} is {verb}ing,* ***Hypothesis:*** *{subject} is {verb}ing,* ***Perturbed Hypothesis:*** *{subject} is {synonym for verb}ing*. This template was applied to both the artifact and basic tests found in the MFT examples above. An additional test was added, in which the same MFT templates were taken, but a random noise sentence was added to the hypothesis, pushing the model to extract the information in a larger amount of text. The perturbed hypothesis in this case would take a form similar to ***Perturbed Hypothesis:*** *A {subject is {verb}ing. {noise sentence}*.

The final set of tests was the DIR tests, designed to test the model's ability to track small changes that push the gold label in a different direction. These required their own templates to be generated, as they required a different structure than the MFT or INV test. The premise remained untouched in the generated examples, but the hypothesis was perturbed. For example, when running the *Entail-Contradict* test, the the original hypothesis would have the label "Entail" while the perturbed hypothesis would contain a negation and shift the label to "Contradiction" following a form similar to ***Premise:*** *A {subject} is {verb}ing,* ***Hypothesis****: A {subject} is {verb}ing,* ***Perturbed Hypothesis:*** *A {subject} is not {verb}ing*. A test beyond the basic and artifact tests was created for this category as well, adding in antonyms for words. Instead of adding a negation term to the examples, the verb in the perturbed hypothesis would be swapped with its antonym, implying a label flip. The antonym tests were only run on tests containing one entailment hypothesis and one neutral hypothesis, as running an antonym flip on a neutral example would have no effect on what the sample was testing.

2.2 Analysis of Base Model

Table 2 contains the performance metrics of the base ELECTRA model on the CheckList tests I generated. Comparing the basic accuracies to the artifact, noise, and antonym accuracies, the majority of tests have higher basic accuracy than their artifact, noise, or antonym counterparts. This behavior is to be expected, as the artifact, noise, and antonym tests are specifically designed to challenge the model in ways the basic test does not. This trend is broken for INV tests on the contradiction label, where the basic accuracy is significantly lower than either the noise or the artifact accuracies. This break in the norm could be due to how the data was generated, and the model confusing the synonym-swapping behavior on the contradict label. If the model did not properly establish a connection between the synonym in the perturbed hypothesis and the original verb in the premise, the semantic similarities between the two broke, collapsing the model's accuracy on those examples. This case is a prime example of the model potentially misunderstanding the semantics of the swap, relying on artifacts rather than the semantics to make its predictions.

The entailment tests provided a key insight into how the model leveraged the negation artifacts to make predictions. While the model performed well at predicting MFT tests in the basic case, the entailment prediction accuracy took a much steeper dive on the artifact tests, dropping from 85.7% to a low 18.8%. The INV entailment tests corroborated with

this performance on the artifact set, with accuracy falling from 95.4% on the basic tests to 37.7% on the artifact test cases. The Contradict-Entail DIR test highlighted a similar trend, with the basic accuracy reaching 59.7% and the artifact accuracy dropping to only 3.8%. These drastic changes in accuracy underscored the model's emphasis on the negation artifacts used in the entailment tests.

The neutral tests are interesting across the 3 test types. The accuracy metric for all tests performed on the neutral label remained well below 50%, with the best accuracy reaching 45.0% and some dipping as low as 0.7%. The poor performance across the board for the neutral tests indicated the model's struggle to identify the meaning of the premise-hypothesis pairs. This error was exacerbated in the artifact tests, where the model's performance did not exceed 30% and dipped as low as 0.1%.

The contradiction tests tended to perform the best on the artifact sets. The MFT artifact accuracy for the contradiction test dropped by only 11.1 percentage points compared to the basic accuracy, but it still remained well above 85%. The INV artifact accuracy was 70.9%, the largest artifact accuracy on the INV tests by 33.2 percentage points. The improved performance on these examples demonstrated that the model places less weight on the universal qualifiers than on the negation terms. The accuracy altered by the qualifiers had a much shallower dip compared to that of the negation perturbations.

Testing the model's understanding of antonyms in the DIR test provided further insight into its failure to grasp the semantics of the examples provided. The accuracy didn't surpass 20% for either of the antonym test sets. The model failed to establish the connection from the verb to the antonym and to capture the direction change being tested.

The model exhibits clear signs of bias in the test results, and requires enhanced training architecture to ensure the performance of the model improves on out-of-test data.

3 Model-Level Debiasing

3.1 Overview

In my testing, model-level debiasing was often an effective technique for creating a more robust NLI model. To debias at the model level, I trained a bag-of-words (BOW) sub-model alongside the ELECTRA model, then projected the ELECTRA model's results orthogonal to those of the BOW sub-model. The BOW sub-model captured simple lexical features from the data that are discouraged in the ELECTRA model through the use of an orthogonal projection. This created a model that relies less on superficial features in the training data and more on complex relationships between premise-hypothesis pairs. At prediction time, the BOW model output was discarded and only the ELECTRA model was used to generate predictions. With the debiased weights, the ELECTRA model became more robust and relied less on artifacts and more on learned features, improving accuracy on out-of-test data for many cases. (Zhou and Bansal, 2020).

3.2 Bag-of-Words Sub-Model

The BOW sub-model utilized a simple feature extractor based on word occurrences to build out the representation of the premise-hypothesis pairing. The full training set was used to generate the feature space, and feature vectors were created for each premise and hypothesis separately. These representations were passed through an embedding layer, ignoring word positions, to obtain embedded representations that could then be fed into a self-attention layer. Each premise and hypothesis had its own self-attention layer, whose attention outputs were then pooled to produce a single vector representation for each premise and hypothesis using the formula $\frac{1}{l}\{self\_attn(w)\}$. These representations were then combined into a single vector for the example, following the formula: $[p, h, p-h, p \odot h]$, where p denotes the premise representation, and h denotes the hypothesis representation. This vector was passed through a simple multilayer perceptron (MLP) to get the final representation of the premise-hypothesis pair for the BOW sub-model.

3.3 HEX Projection

The HEX projection architecture is the most important component in debiasing the large pretrained model (Wang et al., 2019). The formula takes the ELECTRA representation u_{main} and the BOW sub-model representation u_{bow} and creates three new representations to pass to the hex projection formula:

$$F_A = f([u_{bow}, u_{main}|\xi])$$

$$F_P = f([0, u_{main}|\xi])$$

$$F_G = f([u_{bow}, 0|\xi])$$

Test	Type	ELECTRA Acc	*L* Debiased Acc	*S* Debiased Acc
MFT	Entail	**85.7**	**85.7**	**85.7**
	Neutral	45.0	50.6	**52.6**
	Contradict	**98.4**	97.1	96.8
INV	Entail	**95.4**	53.2	56.5
	Neutral	9.9	48.0	**51.5**
	Contradict	23.9	63.0	**79.2**
DIR	Entail-Neutral	10.0	21.0	**30.0**
	Entail-Contradict	67.9	99.4	**99.7**
	Neutral-Entail	1.1	34.5	**53.4**
	Neutral-Contradict	0.7	34.5	**53.4**
	Contradict-Entail	59.7	**100.0**	99.4
	Contradict-Neutral	5.7	8.8	**15.9**

Table 3: Accuracy scores from the Basic Test

These training representations are passed into the projection formula:

$$F_L = (I - F_G(F_G^T F_G)^{-1} F_G^T) F_A$$

Using the new projections F_L and F_G, the BOW-only representation, we feed them both through the same MLP to obtain logits for each. We compute the loss for each of the logits and optimize a weighted combination of the two losses during training. During inference time, we simply use the main model representation F_P only, and ignore the representations involving the BOW sub-model. Predictions are generated using only the main model, which was tuned using the debiased HEX projection during the training process, thereby improving the features the model learns from the corpus.

4 Analysis of Debias

4.1 Model Structure Comparison

The model-level debiasing architecture provides flexibility in the size and shape of the MLP layers used throughout the training process. For this analysis, I altered the sizes of the hidden layers and trained two different debiased models: ***L* Debiased** and ***S* Debiased**. The larger model, ***L* Debiased**, has hidden layer sizes that are twice those of the smaller model, ***S* Debiased**. Doubling the layer sizes increased the representation capacity of the BOW sub-model and the HEX projection algorithm, as both use MLPs to generate outputs. Each model was trained for 5 epochs, ensuring sufficient learning time. The results of these models are recorded in Tables 3-6, each of which contains the results from a particular test category (basic, artifact, noise, or antonym).

4.2 Basic Test Results

The accuracy scores for the basic test results in Table 3 show that the performance of the ***S* Debiased** was the best for nearly each test type. The small debiased model was particularly impressive on the DIR section of tests, showing large jumps in accuracy from the ELECTRA model to the small debiased architecture. Test types like neutral-to-entailment and neutral-to-contradiction were extremely challenging for the base model, with accuracies under 2% for both, whereas the debiased model saw a jump in performance to over 50% accuracy for each. This large performance gain for the ***S* Debiased** model was a consistent trend across many of these basic test types. The neutral INV test jumped from 9.9% accuracy to 51.5% accuracy, and the entail-to-contradiction DIR test experienced a jump from 67.9% accuracy to 99.7% accuracy. In two cases where the ***S* Debiased** model was outperformed, the difference in performance was less than 2 percentage points, and the accuracy of the model was well over 95% in each case. The only test the small debiased model struggled on, compared to the base ELECTRA model, was the entailment INV test, where the base model reached an accuracy of 95.4%, while the ***S* Debiased** model only reached 56.5%. This instance illustrated how the more complex feature space learned by the debiasing architecture can harm performance. The model likely overcomplicates the basic invariance tests and picked up incorrect behavior from its more advanced feature space, resulting in lower predictive power than the base model. An additional consideration has to be made for the potential use of artifacts in the entailment example. Given the

Test	Type	ELECTRA Acc	*L* Debiased Acc	*S* Debiased Acc
MFT	Entail	18.8	30.1	**30.6**
	Neutral	29.5	29.7	**43.2**
	Contradict	**87.3**	69.1	51.9
INV	Entail	**37.7**	22.7	19.9
	Neutral	6.7	24.4	**37.0**
	Contradict	**70.9**	57.3	44.1
DIR	Entail-Neutral	0.5	29.5	**34.3**
	Entail-Contradict	**45.5**	27.3	29.5
	Neutral-Entail	**0.1**	0.0	0.0
	Neutral-Contradict	0.2	**3.8**	2.2
	Contradict-Entail	3.8	**14.8**	3.8
	Contradict-Neutral	0.1	**1.2**	0.0

Table 4: Accuracy scores from the Artifacts Test

simplicity of the basic test cases, the model may exploit artifact usage to achieve a high degree of accuracy on these test cases. This idea is demonstrated in Table 2, where the entailment INV test saw a drop of accuracy from 95.4% to 37.7% when presented with examples designed to exploit artifact usage.

A comparison of each debiasing architecture was warranted to determine the optimal choice for improving the base model. Both the ***S* Debiased** and the ***L* Debiased** models outperformed the ELECTRA model on the majority of these basic test cases. While the ***S* Debiased** model had the best performances on the majority of categories, the ***L* Debiased** model still saw increases in accuracy compared to the ELECTRA model. The smaller model likely outperformed the larger one as the feature space it is required to learn is not nearly as wide, making it simpler to pick up the superficial features and train them out of the model. While the larger debiasing network can learn more complex features, the main theory behind the use of these debiasing networks is to learn the superficial features and prevent the base model from using them as shortcuts in prediction. Overall, the debiasing architecture outperformed the ELECTRA base model on the basic test cases, with the ***S* Debiased** model achieving the best performance.

4.3 Artifact Test Results

The results from the artifact tests in Table 4 show much more variable performance across the three models than in the basic tests. The ***S* Debiased** model still saw some performance gains when compared to the ELECTRA model, but not nearly the dominance it had in the basic tests. The small model improved accuracy scores in the entailment and neutral MFT tests by significant margins, raising the entailment score from 18.8% up to 30.6%. The ***S* Debiased** model also saw a major improvement in the entailment-to-neutral DIR test, achieving an accuracy of 34.3% while the ELECTRA model sits at only 0.5%. The small debiased model appeared to debias the negation examples fairly well, as shown in the 30.6% accuracy on the entailment MFT tests. The debiasing architecture also improved the neutral MFT and INV scores, demonstrating an impressive improvement in semantic understanding compared to the base ELECTRA model. These improvements demonstrate the power that model-level debiasing can have on creating more robust models by learning a better feature space during model training.

The full table results, however, also show possible limitations of the debiasing architecture. The ELECTRA base model outperformed both the ***S* Debiased** model and the ***L* Debiased** model in a handful of tests. The contradict MFT test saw the ELECTRA model reach 87.3% accuracy, while the next-closest model, the ***L* Debiased** model, reached only 69.1%. Similarly, the contradict INV test placed the ELECTRA model over 13 percentage points higher than the next-closest model, the ***L* Debiased** model. The lower performance of the debiased models highlights the limitations even a more robust feature space can have when predicting challenging scenarios. This more complicated feature space requires fine-tuning model parameters to create the optimal debiased architecture for predicting these edge cases. The current ***S* Debiased**

Type	ELECTRA Acc	*L* Debiased Acc	*S* Debiased Acc
Entail	**46.9**	1.2	7.0
Neutral	2.5	29.0	**33.8**
Contradict	33.6	**89.9**	66.7

Table 5: Accuracy scores from the Noise Invariance Test

Type	ELECTRA Acc	*L* Debiased Acc	*S* Debiased Acc
Entail-Contradict	15.7	**36.5**	25.6
Contradict-Entail	**4.5**	3.2	2.6

Table 6: Accuracy scores from the Antonym Directional Expectation Test

and ***L* Debiased** models are not optimally tuned and could therefore undergo small tweaks in an attempt to further improve performance on these challenging test sets.

Striking a balance between complexity and performance for the debiased architecture is one of the main challenges in improving model robustness. Comparing the ***S* Debiased** model to the larger ***L* Debiased** model, the artifact tests show that the wider and more complex feature space created a more evenly distributed performance compared to the smaller model, trading off which test a model performed better at. The larger debiased model saw the majority of its gains within the DIR tests, where it outperformed the smaller model 3 tests to 2. The larger debiased network also saw performance gains over its smaller counterpart for the contradiction MFT test and the entailment INV test. The larger ***L* Debiased** model did not achieve an accuracy over 15% on those tests where it outperformed both other models, showing that the complex nature of those tests may require even more complex feature spaces to model appropriately.

4.4 Noise and Antonym Analysis

The noise INV tests and antonym DIR tests found in Tables 5 and 6 show a similar trend to the artifact tests' results. No model clearly outperformed another, with each performing better on a handful of tests on which another model struggled. The ELECTRA model outperformed the next-closest model by over 35 percentage points on the entailment noise test. The small debiased model achieved the highest accuracy on the neutral noise test, at 33.8%. The larger debiased model performed best on the contradiction noise test, achieving an accuracy of 89.9%.

The antonym tests saw each model struggle overall, with the ELECTRA model achieving the highest accuracy in the contradiction-to-entailment antonym test at 4.5%. The large debiased model, which was best for the entailment-to-contradiction antonym test, achieved an accuracy of 36.5%, outperforming even the small debiased model. When presented with antonyms in these test cases, each model struggled to understand the semantics of the sentences it was presented with, and thus, the accuracy suffered. The debiased architecture showed mixed results on improving the original model's performance, demonstrating the need for an improved debiasing architecture to enhance performance on these challenge sets.

5 Conclusion

5.1 Limitations and Next Steps

The analysis in this paper highlights the potential of a model-level debiasing technique for creating robust natural language inference models. The analysis in this paper covered a couple of variations on a single debiased architecture, focusing on testing the complexity of the feature space learned through the debiased network and leaving the depth of the MLPs alone. Future analysis should be conducted by experimenting with differences in the debiased framework, training epochs, and weighted combination of the F_L and F_G loss discussed in the HEX projection algorithm. The limitations in computing power and time created restraints on training too large a debiased network, so exploring those parameters and MLP sizes could prove interesting in the analysis of the model-level debiasing technique.

Another key area of improvement and limitation is in the CheckList data generation. With limited resources, the script used to generate the data in this analysis could be improved in its diversity to check an even broader range of artifacts the model may have used to shortcut predictions. Each test was

hand-crafted and filled in programmatically to create a fairly sizable set of test examples. Increasing the complexity of the templates beyond the subject-to-verb examples would diversify the analysis on the performance of the ELECTRA model.

The final area of limitation I want to discuss is the pretrained model limitation. To further this analysis, I would not only improve the debiased architecture but also experiment with base models other than the ELECTRA-Small-Discriminator. The smaller ELECTRA model proved a good test of the capabilities of the debiasing solution and was quicker to train than a larger model would be; however, the larger model would be able to learn more complex feature spaces, potentially improving the impact the debiasing architecture would have on the model's overall robustness (Clark et al., 2020). Further testing in this area would be an excellent way to determine the true effectiveness of the model-level-debiasing technique.

5.2 Final Analysis

Pretrained models on natural language inference tasks, such as entailment prediction, can often exhibit inflated scores due to artifacts learned during training. Employing debiasing solutions can produce more robust models and improve performance on challenging out-of-test data. The model-level debiasing technique implemented in this paper demonstrated the ability of a small bag-of-words sub-model to improve the larger pretrained model's performance on difficult test cases. The model-level debiasing pushed the ELECTRA model to learn a more complex feature space, improving performance on many tests but causing performance dips in a handful of others. The complex feature space requires fine-tuning model parameters to optimize the training framework and produce the best overall training results. While the debiasing networks in this analysis did not improve every test's performance, they did make noticeable improvements to the original model's performance, underscoring the importance of removing bias in natural language tasks. Model-level debiasing is a powerful tool and can vastly improve the effectiveness of large pretrained language models.

References

Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics.

Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. 2020. Electra: Pre-training text encoders as discriminators rather than generators. *Preprint*, arXiv:2003.10555.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of nlp models with checklist. *Preprint*, arXiv:2005.04118.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention is all you need. *Preprint*, arXiv:1706.03762.

Haohan Wang, Zexue He, Zachary C. Lipton, and Eric P. Xing. 2019. Learning robust representations by projecting superficial statistics out. *Preprint*, arXiv:1903.06256.

Xiang Zhou and Mohit Bansal. 2020. Towards robustifying nli models against lexical dataset biases. *Preprint*, arXiv:2005.04732.